

Agent Hamster Wheel: Branch-Native Evaluation of Iterative AI Code Review for Local-First Application Engineering

Hyunsoo Lee

May 18, 2026

Abstract

AI coding agents are increasingly used as iterative maintainers: developers ask them to review an existing system, repair defects, rerun checks, and record follow-up findings. Standard repair benchmarks usually collapse this process into a final pass/fail outcome, obscuring which engineering qualities a trajectory actually accumulates. This paper presents Agent Hamster Wheel, a branch-native case study of three model-labeled repair trajectories for the same Flutter local-first notes application. Starting from `codebase-flutter`, each result branch was produced by 10 repeated review prompts and then audited through git metrics, reproduced `flutter analyze` and `flutter test`, test-suite structure, branch logs, and code inspection. All final branch states pass the same quality gates, but they differ sharply: Composer 2.0 concentrates on reactive UI and stream polish, GPT-5.5 repairs FTS5 lifecycle and nullable-domain defects with compact coverage growth, and Sonnet 4.6 accumulates the broadest correctness envelope, including FTS5 apostrophe behavior, domain equality, constructor invariants, and 37 passing tests. The contribution is not a global model ranking; it is an artifact-centered protocol for evaluating iterative agentic maintenance as a software engineering process.

1 Introduction

Modern code-agent evaluation often treats a model as a one-shot solver: a benchmark issue is supplied, a patch is generated, and success is scored by tests or human acceptance. SWE-bench exemplifies the value of this repair-oriented methodology for real GitHub issues [3]. Real maintenance work, however, rarely ends after one patch. Engineers ask for a second review, probe edge cases, rerun tests, inspect logs, and decide whether the next change improves the artifact or merely adds churn.

Agent Hamster Wheel studies that iterative setting in a deliberately small but technically concrete system: a Flutter notes application backed by Drift and SQLite FTS5 [1, 2, 5]. The application is local-first in the sense of Kleppmann et al. [4]: state is owned locally, reactive views derive from local persistence, and correctness depends on database behavior across opens, migrations, stream subscriptions, and full-text-search maintenance.

The research question is:

When different model-labeled agents repeatedly review and repair the same local-first application from the same base branch, what kinds of engineering quality do their branches accumulate?

This paper makes four contributions. First, it defines a branch-native protocol for studying repeated agent maintenance without reducing the result to a scalar leaderboard. Second, it applies the protocol to `codebase-flutter`, `result-flutter-composer-2-0`, `result-flutter-gpt-5-5`, and `result-flutter-sonnet-4-6`. Third, it separates reproduced evidence from interpretive evidence: git-derived metrics, local quality gates, test declarations, branch logs,

and reviewer inspection are reported as distinct signals. Fourth, it extracts a local-first defect taxonomy covering FTS5 repair, reactive stream ownership, nullable domain semantics, test teardown, and domain-model contracts.

2 Related Work

Repair benchmarks have made code-agent progress measurable by grounding tasks in real repositories and executable checks. Their strength is also their limitation for the present question: final acceptance says little about how a branch arrived there, which defects were discovered late, which fixes remained untested, or whether an agent revised its own earlier work. Agent Hamster Wheel complements issue-resolution benchmarks by treating branch history and development logs as evidence rather than discarding them as process noise.

The target domain also matters. Local-first applications must preserve durable local state while exposing responsive reactive views. In this setting, defects often appear outside ordinary create-read-update-delete paths: an FTS5 external-content index can become stale after trigger drift, a migration can omit repair work for already-created databases, and a widget test can hang because a reactive database stream schedules teardown asynchronously. These properties make a local-first data layer a useful stress test for iterative maintenance.

3 Benchmark Artifact

The benchmark subject is `flutter_local_first`, a Flutter package. The implementation uses Drift as a typed persistence layer over SQLite, with native FTS5 search for note title and content. The relevant code-level components are:

- `AppDatabase`, which declares Drift tables for folders, notes, and note tags, sets schema version 2, enables foreign keys, and installs the FTS5 virtual table `fts_notes`.
- `NotesLocalRepository` and `FoldersLocalRepository`, which provide asynchronous operations and stream-based list APIs for UI binding.
- `Note` and `Folder`, the domain models whose nullable parent fields expose the copy-semantics defects discussed in Section 6.
- `LocalFirstNotesApp`, the Flutter shell used by widget tests to exercise the real app boundary rather than a synthetic `MaterialApp` placeholder.

The standardized prompt asks an agent to act as a principal engineer and implement a production-ready local-first data layer for a selected stack. The repository includes prompt tooling for Flutter, React Native, and Tauri trials, but this paper studies only the Flutter trial. The tracked Flutter implementation is branch-resident: `codebase-flutter` contains the common baseline, and the result branches contain the model-labeled trajectories. The active `main` checkout is therefore not treated as the benchmark subject; reproducibility depends on checking out or creating worktrees for the experiment branches.

4 Experimental Design

4.1 Branches And Process

The experiment is encoded directly in git. `codebase-flutter` is the shared baseline. `result-flutter-composer-2-0`, `result-flutter-gpt-5-5`, and `result-flutter-sonnet-4-6` are final result branches named after the agent model used in the corresponding maintenance trajectory. Each result branch records 10 review prompts in `DEVELOPMENT_LOG.md`; `result-flutter-`

Trajectory	Prompts	Commits	Files	Insertions	Deletions	Tests Passed
Base	0	0	–	–	–	2
Composer 2.0	10	10	6	649	166	5
GPT-5.5	10	10	10	635	122	6
Sonnet 4.6	10	11	14	1244	193	37

Table 1: Branch-level outcomes relative to `codebase-flutter`. All listed final states passed `flutter analyze` and `flutter test` when reproduced from worktrees on May 18, 2026.

`sonnet-4-6` has 11 commits rather than 10 because one additional commit rennumbers or consolidates the log.

This evidence establishes branch protocol and artifact state, not cryptographic model provenance. Accordingly, the analysis refers to the result branches as model-labeled trajectories and avoids claims about universal model superiority.

4.2 Measurements

The study uses four evidence channels:

1. **Reproduced gates:** `flutter analyze` and `flutter test` were run from detached worktrees on May 18, 2026.
2. **Git metrics:** commits since `codebase-flutter`, changed files, insertions, deletions, and net line change were computed from branch diffs.
3. **Test artifacts:** test files, `test(...)` declarations, `testWidgets(...)` declarations, and `group(...)` declarations were counted from the Dart test tree on each branch.
4. **Qualitative inspection:** final code and `DEVELOPMENT_LOG.md` entries were inspected for named local-first correctness dimensions.

The first three channels are reproducible from repository commands. The fourth is an expert review judgment constrained by code and tests, and Table 3 should be read with that evidentiary boundary in mind.

4.3 Scoring Philosophy

The protocol intentionally rejects a single scalar leaderboard. A branch can pass all tests while retaining untested defects, and a larger diff can represent either useful coverage or unnecessary churn. The comparison therefore reports gate success, artifact growth, and defect-class coverage separately. This structure asks not only whether a branch is green, but what kind of maintenance work made it green.

5 Results

Table 1 shows that pass/fail status does not discriminate between the final branches: every final state is analyzer-clean and test-clean. The branches diverge instead in the kind and scale of accumulated maintenance. Composer 2.0 has the smallest file-level change set and grows the suite from 2 to 5 tests. GPT-5.5 touches more files and reaches 6 tests while concentrating on database lifecycle repair. Sonnet 4.6 has the largest diff and expands the suite to 37 tests across domain, repository, FTS query, and widget boundaries.

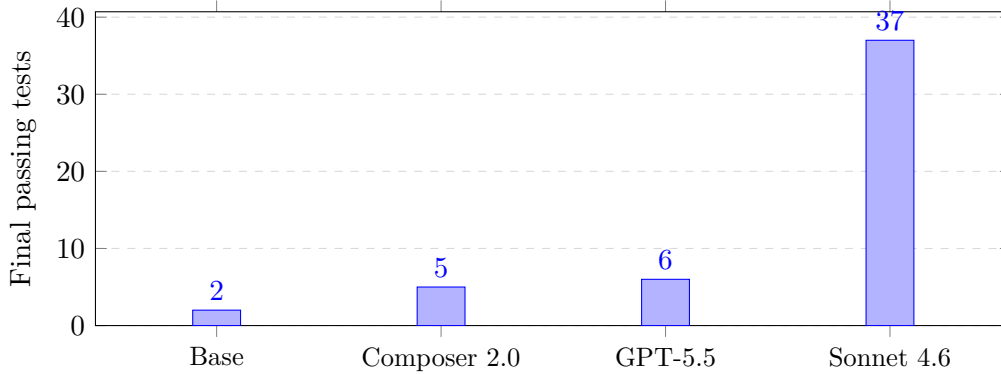


Figure 1: Reproduced final test counts. All branches also passed `flutter analyze`; the figure visualizes test-suite breadth rather than gate success alone.

Trajectory	Test Files	Test Declarations	Groups	Net LOC Change
Base	2	2	0	0
Composer 2.0	2	5	0	483
GPT-5.5	3	6	0	513
Sonnet 4.6	4	37	8	1051

Table 2: Test artifact growth. Net LOC is insertions minus deletions relative to `codebase-flutter`. Test declarations count `test(...)` and `testWidgets(...)` declarations.

Figure 1 and Table 2 show the main quantitative separation: Sonnet 4.6’s larger change set is paired with a much broader explicit test oracle, whereas Composer 2.0 and GPT-5.5 remain close to the baseline test scale.

Table 3 summarizes the qualitative inspection. Composer 2.0 primarily improves reactive UI behavior, stream binding, folder empty states, widget teardown, and FTS hydration reuse. Its residual gaps are technical rather than cosmetic: nullable `copyWith` clearing remains absent, FTS5 delete/update payloads remain under-specified, and apostrophe handling is not covered.

GPT-5.5 concentrates on local-first persistence correctness. Its branch moves FTS5 installation into the database open path, recreates triggers, records a one-time rebuild marker, fixes update/delete payloads, repairs nullable copy semantics through sentinel values, strengthens widget teardown, and replaces default documentation with project-specific architecture notes. This trajectory addresses high-value latent defects, but it stops short of the broader domain-contract and FTS tokenization coverage present in Sonnet 4.6.

Sonnet 4.6 accumulates the broadest engineering envelope. In addition to the FTS5 and nullable-domain fixes also present in GPT-5.5, it adds FTS5 apostrophe-query tests, stream churn reductions, value equality, constructor invariants, repository API symmetry, tag hydration allocation cleanup, and a later correction to keep `CREATE VIRTUAL TABLE` outside an explicit transaction. The last point is especially important for this study: a later pass identified risk in an earlier optimization and repaired the branch, demonstrating self-correction as an observable maintenance behavior.

6 Defect Taxonomy

The branch comparison exposes five local-first defect classes.

Local index correctness. SQLite FTS5 external-content tables depend on correct trigger payloads, trigger recreation, and index rebuild strategy. The baseline installed FTS5 infras-

Quality Dimension	Composer 2.0	GPT-5.5	Sonnet 4.6
Analyzer and test gates	yes	yes	yes
FTS5 reopen and repair path	partial	yes	yes
FTS5 delete/update payload correctness	no	yes	yes
FTS5 apostrophe query behavior	no	no	yes
Nullable <code>copyWith</code> clear semantics	no	yes	yes
Real app widget teardown	yes	yes	yes
Folder empty-state or stream polish	yes	not central	partial
Domain equality and invariants	no	no	yes

Table 3: Qualitative quality matrix from final-branch code inspection and development logs. “Partial” means that a branch addresses part of the concern but leaves a relevant residual gap or treats the issue less comprehensively than another branch.

tructure only during creation and used delete commands that did not include the old indexed title and content values. GPT-5.5 and Sonnet 4.6 repair this class most directly.

Reactive stream ownership. Search and folder streams can churn or diverge from UI state if stream construction is tied to every rebuild or keystroke. Composer 2.0 improves folder stream lifecycle and UI empty states; Sonnet 4.6 also reduces search-stream churn.

Nullable domain semantics. Dart optional parameters using `??` cannot distinguish an omitted argument from an explicit `null`. GPT-5.5 and Sonnet 4.6 introduce sentinel-based `copyWith` methods for nullable parent fields; Composer 2.0 leaves this baseline behavior unchanged.

Test-harness resource safety. Reactive Drift streams can leave pending timers or open database handles in widget tests if the app shell is not unmounted and drained carefully. All three result branches improve the real-app widget boundary, but the logs show different levels of teardown hardening.

Domain model contracts. As the data layer matures, domain objects need equality, hash codes, and invariants. Sonnet 4.6 is the only branch that substantially expands and tests this contract, including empty identifier checks, timestamp ordering for notes, and non-negative folder sort order.

7 Discussion

The results support branch-native evaluation. A final gate score would label every result branch successful. The branch histories reveal a more informative profile: Composer 2.0 exhibits a UI/reactive maintenance bias, GPT-5.5 prioritizes persistence lifecycle correctness, and Sonnet 4.6 pursues broader defect discovery with a much larger test oracle. These differences are not captured by `flutter analyze` and `flutter test` alone.

The local-first domain makes those differences visible. Many important failures are history-dependent rather than simple CRUD failures: indexes go stale after an open or migration

path, nullable fields fail when users clear relationships, and streams misbehave when tests or widgets tear down. Iterative maintenance evaluation should therefore include reopen paths, repair markers, stream ownership, and domain invariants alongside ordinary green tests.

8 Threats to Validity

This is a single-project case study, not a statistically powered model benchmark. The result branches are model-labeled artifacts produced under a repeated-prompt protocol; branch names and logs support the study design but do not independently authenticate model identity. The paper therefore avoids a universal ranking of Composer 2.0, GPT-5.5, and Sonnet 4.6.

The evidence sources also have different strengths. Reproduced quality gates establish final analyzer and test status on May 18, 2026, but they do not prove every intermediate log state-ment. Git metrics are exact for the repository history, while the quality matrix is an expert interpretation constrained by final code, tests, and logs. Finally, the study does not include performance microbenchmarks, blind human review, telemetry, or additional applications; those would be required before making stronger claims about runtime efficiency or general agent quality.

9 Reproducibility

The branch comparison can be reproduced from the repository by inspecting branch diffs and running Flutter checks from isolated worktrees:

```
git branch --all --verbose --no-abbrev
git diff --shortstat codebase-flutter..result-flutter-composer-2-0
git diff --shortstat codebase-flutter..result-flutter-gpt-5-5
git diff --shortstat codebase-flutter..result-flutter-sonnet-4-6

git worktree add --detach /tmp/agent-hw-base codebase-flutter
git worktree add --detach /tmp/agent-hw-composer result-flutter-composer-2-0
git worktree add --detach /tmp/agent-hw-gpt result-flutter-gpt-5-5
git worktree add --detach /tmp/agent-hw-sonnet result-flutter-sonnet-4-6

cd /tmp/agent-hw-sonnet/flutter_local_first
flutter pub get
flutter analyze
flutter test
```

The same Flutter commands should be run inside each worktree’s `flutter_local_first` directory. The reproduced final counts are reported in Table 1; all listed branch states were analyzer-clean.

10 Conclusion

Agent Hamster Wheel demonstrates a practical evaluation pattern for iterative AI software maintenance: freeze a base branch, create one result branch per model-labeled trajectory, preserve development logs, rerun local quality gates, and compare defect classes rather than only final pass/fail status. In this repository, all three result branches pass the same Flutter gates, yet they accumulate distinct kinds of engineering quality. The central methodological lesson is that branch histories and repeated-prompt traces expose maintenance behavior that one-shot benchmark scores hide.

References

- [1] Drift Project. Drift: Reactive persistence library for dart and flutter. <https://drift.simonbinder.eu/>. Accessed 2026-05-18.
- [2] Flutter Project. Flutter documentation. <https://flutter.dev/>. Accessed 2026-05-18.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.
- [4] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. <https://www.inkandswitch.com/local-first/>, 2019. Accessed 2026-05-18.
- [5] SQLite Consortium. Sqlite fts5 extension. <https://www.sqlite.org/fts5.html>. Accessed 2026-05-18.